

VULNERABILITY ASSESSMENT REPORT

For: Future Interns - Cyber Security Task 1 (2026)

Target: Damn Vulnerable Web Application (DVWA)

URL: <http://localhost>

Prepared by: IGNAS EDD MALEWA

CIN ID: FIT/APR26/CS8072

Date: May 8, 2026

1. SCOPE & METHODOLOGY

Assessment Type: Read-only / Passive vulnerability assessment

Target URL: http://localhost

Application: Damn Vulnerable Web Application (DVWA)

Environment: Local Docker container

Allowed Activities

- Public pages only
- Passive scanning
- Header inspection
- Configuration analysis

Not Performed

- Exploitation
- SQL injection testing
- Brute force attacks
- Denial of Service

Tools Used

- Nmap - Port scanning and service detection
- OWASP ZAP - Passive vulnerability scanning
- WhatWeb - Web technology fingerprinting
- Firefox DevTools - Header and cookie analysis
- cURL - HTTP header inspection

2. EXECUTIVE SUMMARY

During this read-only vulnerability assessment, 5 security issues were identified.

Risk Level Distribution

- High Risk: 0 findings
- Medium Risk: 3 findings
- Low Risk: 2 findings

Key Findings Overview

- 1. Missing Content-Security-Policy (CSP) header - Medium
- 2. MySQL database exposed on port 3306 - Medium
- 3. Missing X-Frame-Options header - Medium
- 4. Cookie missing Secure and SameSite attributes - Low
- 5. Server version information disclosure - Low

Priority Recommendations

- 1. Implement security headers (CSP, X-Frame-Options, X-Content-Type-Options)
- 2. Restrict MySQL to listen only on localhost
- 3. Add Secure and SameSite flags to session cookies
- 4. Disable server version disclosure in HTTP headers

3. FINDINGS SUMMARY

Findings	Risk level
Missing Content-Security-Policy (CSP) Header	Medium
MySQL Database Exposed on Port 3306	Medium
Missing X-Frame-Options Header	Medium
Cookie Missing Secure & SameSite Attributes	Low
Server Version Information Disclosure	Low

Each finding is explained one after another clearly below

4. DETAILED FINDINGS

FINDING 1: Missing Content-Security-Policy Header

Risk Level: Medium

Description

The Content-Security-Policy (CSP) header is not set in any HTTP responses from the web application. This header instructs the browser which resources (scripts, styles, images) are allowed to load.

Evidence

WhatWeb scan revealed the server as Apache/2.4.25 (Debian). No CSP header was detected in the HTTP response headers.

Impact

Without CSP, the browser will execute any script loaded by the page, regardless of its source. This makes the application vulnerable to Cross-Site Scripting (XSS) attacks, where attackers can inject malicious scripts into web pages viewed by other users. These scripts could steal session cookies, redirect users, or deface the website.

Remediation

Add the following line to the Apache configuration file (.htaccess or httpd.conf). Header always set Content-Security-Policy "default-src 'self'"

For more restrictive policy, consider the following

Header always set Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self'"

Evidence Files: whatweb.txt, headers.txt

FINDING 2: MySQL Database Exposed on Port 3306

Risk Level: Medium

Description

Nmap network scan discovered that MySQL/MariaDB database service is running and accessible on port 3306. This port was found to be open and responding to connection attempts.

Evidence from Nmap scan

PORT STATE SERVICE VERSION

3306/tcp open mysql MariaDB 11.8.6

- The service fingerprint confirms this is MariaDB version 11.8.6 running on Debian Linux.

Impact

Exposing database services to the network increases the system's attack surface unnecessarily. Remote attackers who gain network access could

- Attempt to brute force database credentials
- Exploit known vulnerabilities in MariaDB 11.8.6
- Access sensitive data if authentication is weak or misconfigured
- Use the database as a pivot point to attack other systems
- Even if firewall rules restrict external access, internal attackers or compromised services on the same host could target this open port.

Remediation

Configure MySQL/MariaDB to bind only to localhost (127.0.0.1), not to all network interfaces (0.0.0.0).

Steps

1. Edit the MySQL configuration file:

```
sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
```

2. Find or add the bind-address directive:

```
bind-address = 127.0.0.1
```

3. Restart the MySQL service:

```
sudo systemctl restart mysql
```

- After this change, only processes running on the same machine can access the database.

Evidence Files: nmap.txt

FINDING 3: Missing X-Frame-Options Header

Risk Level: Medium

Description

The X-Frame-Options HTTP response header is not configured. This header controls whether a browser should be allowed to render a page within a frame or iframe.

Evidence from headers.txt

HTTP/1.1 302 Found

Server: Apache/2.4.25 (Debian)

Set-Cookie: PHPSESSID=...

[No X-Frame-Options header present]

Impact

Without this header, attackers can embed the vulnerable website within a malicious iframe on their own domain. This enables clickjacking attacks, where

1. A user visits the attacker's malicious website
2. The victim website is loaded invisibly inside an iframe
3. The user thinks they are clicking on the attacker's page, but they are actually clicking on hidden buttons or links from the victim website
4. This can lead to unintended actions like changing passwords, making purchases, or transferring funds

Remediation

Add the following header to Apache configuration:

Header always append X-Frame-Options DENY

Evidence Files: headers.txt

FINDING 4: Cookie Missing Secure and SameSite Attributes

Risk Level: Low

Description

The session cookies used by the application lack important security attributes. Specifically, the Secure flag and SameSite attribute are missing from cookie configuration.

Evidence from cookies.txt

localhost FALSE / FALSE 0 security low

localhost FALSE / FALSE 0 PHPSESSID qqm6h4tah8m41h1ila3jn71bq4

Cookie Analysis

- HttpOnly flag: Present (prevents JavaScript access)
- Secure flag: Missing (cookie sent over unencrypted HTTP)
- SameSite attribute: Missing (no CSRF protection)

Impact

Secure Flag Missing

Without the Secure flag, cookies are transmitted over unencrypted HTTP connections. An attacker on the same network could intercept these cookies using packet sniffing (Man-in-the-Middle attack). Once intercepted, the attacker can impersonate the legitimate user by using the stolen session cookie.

SameSite Attribute Missing

Without SameSite protection, the application is vulnerable to Cross-Site Request Forgery (CSRF) attacks. An attacker could trick a logged-in user into clicking a malicious link that performs actions on the victim website without their consent.

Remediation:

- When setting cookies, include all security attributes
- Set-Cookie: PHPSESSID=value; HttpOnly; Secure; SameSite=Strict

Note: The Secure flag only works over HTTPS. If the site does not use HTTPS, implement HTTPS first.

Evidence Files: cookies.txt

FINDING 5: Server Version Information Disclosure

Risk Level: Low

Description

The web server exposes detailed version information in HTTP response headers. This includes the exact Apache version and operating system details.

Evidence from WhatWeb

Server: Apache/2.4.25 (Debian)

Additional information disclosed,

- PHP backend presence detected
- Debian Linux as the operating system

Impact

While not a direct vulnerability, disclosing exact software versions helps attackers during the reconnaissance phase of an attack. Attackers can

1. Identify known vulnerabilities specific to Apache 2.4.25
2. Search for exploits targeting Debian-specific configurations
3. Tailor their attack strategy based on known weaknesses of disclosed versions

This information reduces the attacker's effort required to fingerprint the target and find potential entry points.

Remediation

Hide server version information by configuring Apache to disclose minimal information.

After remediation, the Server header would show only: Server: Apache

Evidence Files: whatweb.txt, headers.txt

5. CONCLUSION AND RECOMMENDATIONS

SUMMARY

This vulnerability assessment was conducted on Damn Vulnerable Web Application (DVWA) running on localhost. The assessment used only passive, read-only techniques with no exploitation.

A total of 5 security issues were identified

- 3 Medium-risk findings requiring attention
- 2 Low-risk findings for defense-in-depth improvement

No critical vulnerabilities were discovered during this assessment.

PRIORITY ACTION PLAN

The following recommendations are listed in order of priority:

1. Add security headers
Implement Content-Security-Policy (CSP) and X-Frame-Options headers in the Apache configuration. The CSP header prevents execution of unauthorized scripts, mitigating XSS attacks. The X-Frame-Options header prevents clickjacking by controlling whether the site can be embedded in frames.
2. Restrict MySQL to localhost only
Configure MySQL to bind only to 127.0.0.1 instead of all network interfaces. This reduces attack surface by preventing remote connections to the database.
3. Add secure and sameSite cookie attributes
Configure session cookies to include Secure and SameSite attributes. The Secure flag ensures cookies are only sent over encrypted HTTPS connections. The SameSite attribute prevents Cross-Site Request Forgery (CSRF) attacks.

Configure Apache to hide detailed version information by setting ServerTokens Prod and ServerSignature Off. This prevents attackers from identifying specific version vulnerabilities during reconnaissance.

ETHICAL COMPLIANCE STATEMENT

I hereby confirm that all testing performed for this assessment was within the defined read-only scope.

- No exploitation was attempted or executed
- No SQL injection or brute force attacks were performed
- No denial-of-service tests were conducted
- All tools were used in passive scan mode only

The target (DVWA) is a deliberately vulnerable web application used exclusively for authorized security training and assessment purposes.

